

Power BI Analytics Agent – Technical Report

Contents

Power BI Analytics Agent – Technical Report	1
1. Purpose	1
2. Architecture Overview	1
3. Engineering Foundation	3
4. LLM Layer	3
5. Parser Layer – the core data model	4
6. Agent Orchestration	4
7. Analysis & Generation Modules	5
7a. Model Inspector – rule-based + LLM	5
7b. Business Documentation Generator	5
7c. PBIP Scaffold – generate a project from raw data	5
7d. Model Fixer – automated remediation with a health score	6
8. Validation Layer	6
9. Design Decisions	6
10. Limitations	7
11. End-to-End Flow: PBIP Input -> AI Agent -> Analysis/Generation -> Validation -> Updated Report	7
12. File Map	7

Power BI Analytics Agent – Technical Report

This report documents the engineering foundation of the Power BI Analytics Agent for reviewers who want to understand how it's built, not just what it does. It covers the architecture, the LLM layer, each core module with code snippets, the design decisions behind them, known limitations, and the end-to-end flow from a raw PBIP project to a validated, improved one.

1. Purpose

The agent connects to data sources (CSV/Excel/SQL Server), parses Power BI project files (PBIP/TMDL format), inspects and reviews them against a rule-based checklist plus an LLM analysis pass, generates plain-language business documentation, scaffolds new PBIP projects from raw data, and can automatically remediate model gaps (missing DAX, misconfigured date tables) with a transparent before/after health score.

Everything that doesn't require an LLM call works fully offline (`--no-llm`); LLM-backed features require an Anthropic API key.

2. Architecture Overview

No live/MCP connection to a running Power BI Desktop session is involved. The agent reads and writes static PBIP project files on disk directly; the user opens the result in Power BI Desktop themselves when ready. All API calls go through the Orchestrator, which routes intent (Haiku) and dispatches to the relevant analysis/generation module, which in turn reads/writes the PBIP project files on disk.

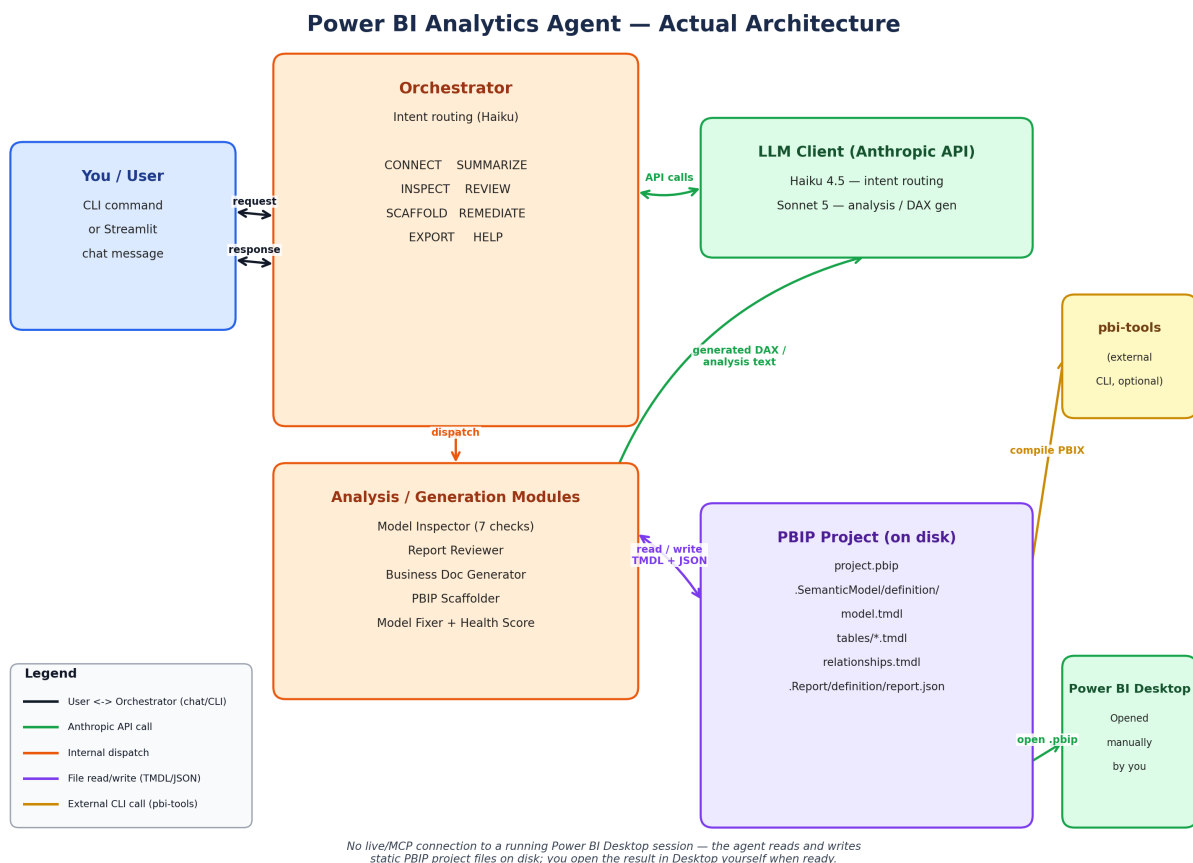


Figure 1: Architecture Diagram

3. Engineering Foundation

Stack: Python 3.10+, Click (CLI), Streamlit (web UI), Anthropic SDK, pandas/openpyxl (data), pyodbc/SQLAlchemy (SQL Server), PyYAML + python-dotenv (config), Rich (console output).

Config system – dataclasses loaded from YAML, overridden by environment variables, so the same code path works whether you're running locally or in CI:

```
# config.py (simplified)
@dataclass
class LLMConfig:
    router_model: str = "claude-haiku-4-5-20251001"
    analysis_model: str = "claude-sonnet-5"
    max_tokens: int = 4096
    api_key: str = "" # populated from ANTHROPIC_API_KEY env var

def load_config(yaml_path: str, env_path: str) -> AgentConfig:
    load_dotenv(env_path) # .env values become os.environ
    raw = yaml.safe_load(Path(yaml_path).read_text())
    cfg = AgentConfig.from_dict(raw) # YAML -> dataclasses
    cfg.llm.api_key = os.environ.get("ANTHROPIC_API_KEY", "")
    return cfg
```

Logging is structured JSON to a file and colored text to console (logging/logger.py), so production log aggregation and local debugging both work from the same call sites.

4. LLM Layer

Two models, deliberately different jobs:

Model	Role	Why
Claude Haiku 4.5	Intent routing (every chat message)	Fires on every message; needs to be fast/cheap, only outputs one word
Claude Sonnet 5	Analysis, DAX generation, documentation	Needs real reasoning over dozens of tables/measures without hallucinating

```
# llm/client.py -- intent classification
def route(self, user_message: str) -> str:
    system = (
        "Classify the user message into exactly one of these categories:\n"
        "- CONNECT / SUMMARIZE / INSPECT / REVIEW / SCAFFOLD / REMEDIATE / EXPORT / HELP\n"
        "Respond with ONLY the category name, nothing else."
    )
    response = self.client.messages.create(
        model=self.config.router_model,
        max_tokens=20, # one word out -- keep it cheap
        system=system,
        messages=[{"role": "user", "content": user_message}],
    )
    return _extract_text(response).strip().upper()
```

One non-obvious fix worth calling out: newer Claude models can return a **ThinkingBlock** before the actual text response, so `response.content[0].text` isn't a safe assumption. The client scans for the first block

with `type == "text"` instead:

```
def _extract_text(response) -> str:
    for block in response.content:
        if getattr(block, "type", None) == "text":
            return block.text
    return ""
```

5. Parser Layer – the core data model

Input: a PBIP project folder (.pbip manifest + .SemanticModel + .Report subfolders containing TMDL text files and report.json). **Output:** a SemanticModel dataclass tree (tables -> columns/measures/hierarchies/partitions, relationships, roles) and a ReportDefinition (pages -> visuals -> projections/filters).

Real-world PBIP projects use two different on-disk table layouts depending on how they were created – folder-per-table (tables/Sales/definition.tmdl) or flat files (tables/Sales.tmdl). The parser detects and handles both:

```
# inspector/tmdl_parser.py
for entry in sorted(tables_dir.iterdir()):
    tmdl_file = None
    if entry.is_dir():
        candidate = entry / "definition.tmdl"
        if candidate.exists():
            tmdl_file = candidate          # legacy folder-per-table layout
    elif entry.is_file() and entry.suffix == ".tmdl":
        tmdl_file = entry                  # flat-file layout
    if tmdl_file:
        model.tables.append(self._parse_table_file(tmdl_file))
```

Relationships also have two valid syntaxes – separate `fromTable:/fromColumn:` lines, or a single combined `fromColumn: Table.Column` reference (including quoted column names with spaces, e.g. `DimCustomer.'First Order Date'`). Both are normalized to the same Relationship dataclass fields via a small splitter that respects quoting:

```
def _split_table_column(self, value: str) -> tuple[str, str]:
    match = re.match(r"^([\^.']+)\.(.+)$", value.strip())
    if not match:
        return "", value.strip("'\"")
    return match.group(1).strip(), match.group(2).strip().strip("'\"")
```

6. Agent Orchestration

The Orchestrator is a thin router with a session state machine (SessionContext) that tracks the loaded PBIP path, connected data source, and last results across turns of a chat conversation:

```
# orchestrator.py
def process(self, user_message: str) -> str:
    intent = self.llm.route(user_message)          # Haiku classifies the message
    try:
        match intent:
            case "CONNECT": response = self._handle_connect(user_message)
            case "SUMMARIZE": response = self._handle_summarize(user_message)
            case "INSPECT": response = self._handle_inspect(user_message)
            case "REVIEW": response = self._handle_review(user_message)
            case "SCAFFOLD": response = self._handle_scaffold(user_message)
```

```

        case "REMEDiate": response = self._handle_remediate(user_message)
        case "EXPORT":     response = self._handle_export(user_message)
        case "HELP" | _:  response = self._handle_help(user_message)
    except Exception as e:
        response = f"Error during {intent.lower()}: {e}"
    return response

```

Modules (connection_manager, model_inspector, report_reviewer, pbip_scaffolder, model_fixer, export_engine) are all @property-lazy – constructed on first use, not at startup, so a pure --no-llm CLI session never imports the anthropic package.

7. Analysis & Generation Modules

7a. Model Inspector – rule-based + LLM

Runs seven deterministic checks (documentation coverage, relationship validity, date-table configuration, measure health, column typing, RLS role sanity, naming conventions) before optionally asking Claude for a deeper pass:

Input: a PBIP path. **Output:** a structured `InspectionResult` (severity-tagged findings + optional LLM narrative).

```

def inspect(self, pbip_path: str, use_llm: bool = True) -> dict:
    model = self.parser.parse_project(pbip_path)
    result = InspectionResult(model_summary=model.to_summary_dict())
    self._check_documentation(model, result)
    self._check_relationships(model, result)
    self._check_date_table(model, result)
    self._check_measures(model, result)
    # ... rule-based, deterministic, no API call
    if use_llm and self.llm:
        result.llm_analysis = self._llm_analyze(model, result) # optional Sonnet pass
    return result.to_dict()

```

7b. Business Documentation Generator

Input: parsed `SemanticModel` + `ReportDefinition`. **Output:** a 12-section markdown document written for non-technical stakeholders (project overview, data tables, relationships, DAX measures explained in plain English, report pages, filters, security, data sources, key metrics, navigation, executive summary).

Design constraint baked into the system prompt: the model must only describe what’s actually present in the extracted JSON and explicitly say when something (e.g. refresh schedule, bookmarks) can’t be determined from the data – no invented facts.

7c. PBIP Scaffolder – generate a project from raw data

Input: a connected CSV/Excel file’s extracted schema (`FileConnectionResult`: table name, columns, inferred types, sample values). **Output:** a starter .pbip project – TMDL table definitions with a Power Query source step and a minimal report page.

```

def _build_m_expression(self, table: TableMeta) -> tuple[str, list[str]]:
    if table.file_type == "csv":
        return (
            'let\n'
            f'    Source = Csv.Document(File.Contents("{table.source_path}"), '\n'
            f'    [Delimiter=",", Columns={len(table.columns)}, Encoding=65001]),\n'
            '    #"Promoted Headers" = Table.PromoteHeaders(Source, [PromoteAllScalars=true])\n'

```

```

        'in\n      # "Promoted Headers"
    ), []
    # Excel branch resolves the first sheet via pandas.ExcelFile -- see limitations

```

7d. Model Fixer – automated remediation with a health score

Input: an existing PBIP path. **Output:** a fixed copy of the project (original untouched by default) + a HealthScore before/after + a list of every change made.

Health score is a transparent, documented formula – not a black box:

```

# remediate/health_score.py
def calculate_health_score(counts: dict) -> HealthScore:
    raw = 100.0 - (counts["errors"] * 8.0) - (counts["warnings"] * 3.0) - (counts["info"] * 0.5)
    return HealthScore(score=max(0.0, min(100.0, raw)), **counts)

```

Empty measures are batched into a single LLM call (not one call per measure – cheaper and gives the model full context of what columns/measures already exist), and every generated formula is tagged so nobody mistakes a guess for verified business logic:

```

measure.description = f"[GENERATED -- VERIFY] {explanation}"

```

The fixed model is serialized back to TMDL with a dedicated writer (`remediate/tmdl_writer.py`) rather than string-patching the original file – this regenerates each table file deterministically from the in-memory dataclass, which is safer than trying to find-and-replace inside hand-written TMDL text.

8. Validation Layer

Every writer in this codebase (scaffolder, model fixer) is validated the same way before being trusted: **write, then re-parse, then assert the structure matches expectations**. This round-trip check was run during development for every generator – e.g., after `ModelFixer` writes a fixed `DimDate.tmdl`, the test suite re-parses it with the same `TMDLParser` used for inspection and confirms `dataCategory == "Time"`, the hierarchy exists, and the measure DAX is non-empty. This catches serialization bugs (wrong indentation, broken quoting) before they ever reach a real project.

9. Design Decisions

- **Copy-by-default remediation.** `ModelFixer.remediate()` writes to a new `_fixed` folder unless `in_place=True` is explicitly passed. Auto-rewriting DAX and model structure is powerful enough that “never touch the original by accident” outweighs convenience.
- **Generated DAX is flagged, not silent.** Every LLM-authored measure gets a `[GENERATED -- VERIFY]` tag and a one-sentence explanation of the assumption made, because the tool has no way to know your actual business definition of “churn” or “retention” from column names alone.
- **Report pages are a specification, not an injection.** The report parser reads 0 pages from real-world Power BI Desktop report files (their on-disk schema has more version-specific detail than this tool’s simplified model can safely round-trip). Rather than risk writing invalid JSON into a real report file, the remediation feature hands back a structured page/visual spec for a human (or Power BI Desktop) to build from.
- **Two-tier LLM routing.** Haiku for one-word classification, Sonnet for everything requiring real reasoning – keeps chat responsiveness high and cost proportional to task complexity.
- **Offline-first core.** Inspect/Review/Connect/Export all function without an API key; LLM analysis is additive, not required.

10. Limitations

- **Report-page writing isn't fully supported** for real Power BI Desktop projects – see design decision above. Newer PBIR-format reports (separate page/visual JSON files) aren't parsed at all yet; only the older monolithic `report.json` schema is.
- **Generated DAX is best-effort**, inferred from measure/column naming conventions. It is not validated against your actual business rules and must be reviewed before use in production reporting.
- **PBIX binary format isn't parsed directly** – PBIX export/extraction wraps the external `pbi-tools` CLI rather than handling the binary format natively, so `pbi-tools` must be installed for those features.
- **Excel sheet detection uses the first sheet only** when scaffolding a new PBIP from a multi-sheet workbook; other sheets are ignored unless connected individually.
- **The health score is a custom heuristic** (weighted error/warning/info counts), not an implementation of Microsoft's official Best Practice Analyzer rule set – it's directionally useful, not a certified score.
- **No RLS (row-level security) logic testing** beyond structural checks (does the role reference a real table, is the filter expression non-empty) – the actual security *behavior* of a role's DAX filter isn't evaluated.
- **Scaffolded PBIP projects aren't guaranteed byte-perfect** for Power BI Desktop; they're validated against this tool's own parser, and generally need a "Transform Data" check on first open in Desktop.

11. End-to-End Flow: PBIP Input -> AI Agent -> Analysis/Generation -> Validation -> Updated Report

[1] PBIP INPUT	[2] AI AGENT	[3] ANALYSIS/GENERATION
----- .pbip + .SemanticModel -> (TMDL text) + .Report (report.json)	----- Orchestrator.process() -> +- Haiku routes intent +- dispatches to handler	----- Rule-based checks (7 checks, no LLM) produce baseline findings + Health Score -> Sonnet (optional) adds: narrative analysis, generated DAX for empty measures, business documentation
[4] VALIDATION		[5] UPDATED REPORT
----- Write fixed TMDL -> re-parse -> with same parser -> confirm structure (dataCategory, hierarchy, non-empty DAX) before returning success		----- _fixed/ project folder + health score (before/after) + dashboard page spec + change log, original untouched unless --in-place

Concretely, for the `remediate` command:

1. **Input:** path to a real PBIP project.
2. **Agent:** `Orchestrator` (or CLI) routes to `ModelFixer.remediate()`.
3. **Analysis/Generation:** rule-based `ModelInspector` computes the *before* score; `ModelFixer` fixes the date table structurally, then sends all empty measures in one batched call to Sonnet for DAX generation.
4. **Validation:** the fixed project is written to disk, then re-parsed with the same `TMDLParser` used everywhere else in the tool to compute the *after* score – if the write were malformed, this step would surface it immediately as a parse failure rather than a silent corrupt file.
5. **Updated report:** a `_fixed` project folder (original untouched by default), a before/after health score, a full change log, and a dashboard page specification for the two new report pages.

12. File Map

`src/pbi_agent/`

config.py	# dataclass config, YAML + env merge
orchestrator.py	# intent routing, session state
llm/client.py	# Haiku/Sonnet wrapper
connectors/	# CSV/Excel + SQL Server
inspector/	
tmdl_parser.py	# core TMDL + report.json parser
model_inspector.py	# 7 rule-based checks + LLM pass
business_doc.py	# 12-section documentation generator
reviewer/report_reviewer.py	# report health checks
export/	
export_engine.py	# pbi-tools wrapper (PBIX compile/extract)
pbip_scaffolder.py	# generate PBIP from CSV/Excel
remediate/	
health_score.py	# transparent scoring formula
tmdl_writer.py	# Table dataclass -> TMDL text
model_fixer.py	# date table + DAX remediation
main.py	# CLI (Click)
app/streamlit_app.py	# web UI